CreditPathAI Project – Data Cleaning Pipeline Handout

This handout explains the data cleaning stage of the CreditPathAI Smart Loan Recovery System project. After the raw dataset has been safely ingested into PostgreSQL, the next step is to extract the dataset into Python and prepare it for analysis and machine learning. Data cleaning is one of the most important steps in any machine learning pipeline. Real-world datasets often contain missing values, duplicate records, inconsistent datatypes, and categorical values that cannot be directly used by machine learning algorithms. If these issues are not addressed, models may produce inaccurate predictions or fail during training. The objective of this stage is to transform raw data into a reliable and structured dataset that can be safely used for exploratory analysis and model training.

Step 1 – Connect Python to PostgreSQL

First, install the required Python libraries: pip install pandas psycopg2 sqlalchemy Then connect to the PostgreSQL database and load the dataset. Example code: import pandas as pd from sqlalchemy import create_engine engine = create_engine("postgresql://postgres:password@localhost:5432/creditpath") df = pd.read_sql("SELECT * FROM loan_default", engine) Reason for this step: Machine learning workflows operate on data structures such as Pandas DataFrames. Although the dataset is stored in PostgreSQL, it must be loaded into Python so that we can perform cleaning operations, transformations, and analysis.

Step 2 – Inspect Dataset Structure

After loading the dataset, examine the structure of the DataFrame. Example code: df.info() This command displays column names, datatypes, and the number of non-null values. Reason for this step: Before modifying data, we must understand the structure of the dataset. Inspecting the dataset allows us to verify that columns were imported correctly and identify potential issues such as missing values or incorrect datatypes.

Step 3 – Identify Missing Values

Check for missing values using: df.isnull().sum() If missing values exist, they must be handled before model training. Example: df['income'].fillna(df['income'].median(), inplace=True) Reason for this step: Missing values can cause machine learning algorithms to fail or produce biased predictions. Filling missing values using statistical methods such as the median ensures that the dataset remains complete and usable.

Step 4 – Detect Duplicate Records

Check whether duplicate rows exist: df.duplicated().sum() If duplicates are found: df = df.drop_duplicates() Reason for this step: Duplicate records distort statistical distributions and bias machine learning models. Removing duplicates ensures that each record represents a unique borrower in the dataset.

Step 5 – Verify Column Datatypes

Check the datatype of each column: df.dtypes Example expected datatypes: age → integer income → float loan_amount → float credit_score → integer education → object default_status → integer Reason for this step: Incorrect datatypes can cause errors during feature engineering or model training. Ensuring that each column has the correct datatype prevents computational issues later in the pipeline.

Step 6 – Remove Non-Predictive Columns

The column loan_id acts only as an identifier and does not contain useful predictive information for the machine learning model. Example: df = df.drop(columns=['loan_id']) Reason for this step: Identifiers introduce noise into machine learning models. Since each loan_id value is unique, the model cannot learn meaningful patterns from it. Removing such columns improves model reliability.

Step 7 – Detect Outliers in Numerical Data

Examine numerical statistics: df.describe() Visualize distributions using boxplots. Example: import seaborn as sns sns.boxplot(df['income']) Reason for this step: Extreme values (outliers) can distort the learning process of machine learning algorithms. Identifying outliers allows us to either remove them or cap them to maintain stable distributions.

Step 8 – Encode Categorical Variables

Machine learning models require numerical input. Columns containing text categories must therefore be encoded. Example columns: education employment_type marital_status loan_purpose Example encoding: df = pd.get_dummies(df, drop_first=True) Reason for this step: Algorithms cannot interpret textual categories. Encoding converts categories into numerical columns that can be processed by machine learning models.

Step 9 – Scale Numerical Features

Some machine learning algorithms perform better when numerical features are normalized. Example: from sklearn.preprocessing import StandardScaler scaler = StandardScaler() df[['income','loan_amount','credit_score']] = scaler.fit_transform(df[['income','loan_amount','credit_score']]) Reason for this step: Scaling ensures that features with large ranges do not dominate the learning process. Standardization places features on a similar scale so that models can learn more effectively.

Step 10 – Save the Clean Dataset

After completing all cleaning operations, save the processed dataset. Example: df.to_csv("clean_loans.csv", index=False) Reason for this step: Separating raw and cleaned datasets ensures reproducibility. The raw dataset remains preserved in PostgreSQL, while the clean dataset becomes the input for exploratory data analysis and machine learning models.